# WebGPU

## From Zero to Hero

Part I: Theory | Part II: Hands-on JavaScript

9 runnable examples, 8 Graphviz diagrams

### What you will learn

- What WebGPU is, why it exists, and how it differs from WebGL
- How a GPU works and how WebGPU models it (adapter, device, queue, pipelines, bind groups)
- WGSL, the WebGPU shading language: types, memory layout, entry points
- The render pipeline and compute pipeline, step by step
- Hands-on JavaScript: triangle, buffers, uniforms, textures, a 3D cube, compute shaders, GPU particles
- Debugging, performance habits and a roadmap for what to learn next

### How to use this guide

Read Part I once to build the mental model, then type (or download and run) the examples in Part II in order. Each example adds exactly one new idea. All code is plain JavaScript modules with no build step: you only need a browser with WebGPU and a tiny static web server.

# Contents


**1. Introduction: what is WebGPU? 6**

1.1 What it is . . . 6
1.2 Why was it created? . . . 6
1.3 WebGL vs WebGPU at a glance . . . 6
1.4 Browser support and requirements . . . 7
1.5 What can you build with it? . . . 7

**2. GPU fundamentals 8**

2.1 CPU vs GPU . . . 8
2.2 Threads, groups and lock-step execution . . . 8
2.3 Memory and why copies matter . . . 8
2.4 Two kinds of GPU work . . . 8
2.5 The GPU runs on its own timeline . . . 8

**3. WebGPU architecture 10**

3.1 The software stack . . . 10
3.2 The object model . . . 10
3.3 Features and limits . . . 12

**4. Resources: buffers, textures, samplers 13**

4.1 Buffers . . . 13
4.2 Textures . . . 13
4.3 Samplers . . . 14
4.4 The canvas . . . 14

**5. WGSL: the WebGPU Shading Language 15**

5.1 First look . . . 15
5.2 Types . . . 15
5.3 Control flow . . . 15
5.4 Address spaces (where variables live) . . . 16
5.5 Entry points and attributes . . . 16
5.6 Memory layout rules (very important for buffers) . . . 17
5.7 Useful built-in functions . . . 17

**6. Pipelines and resource binding 18**

6.1 Pipelines . . . 18
6.2 Bind groups and layouts . . . 18

**7. The render pipeline in depth** **20**

7.1 Walking through the stages . . . 20
7.2 Coordinate systems . . . 21
7.3 Winding order and culling . . . 21
7.4 The MVP transform . . . 21
7.5 Depth buffer . . . 21
7.6 Render pass: attachments, load and store . . . 21
7.7 Instancing . . . 22

**8. The compute pipeline in depth** **23**

8.1 Workgroups . . . 23
8.2 Choosing a workgroup size . . . 23
8.3 Workgroup memory and barriers . . . 23
8.4 Reading results . . . 24
8.5 Race conditions and atomics . . . 24

**9. Command encoding and data flow** **25**

9.1 Command encoder . . . 25
9.2 Moving data . . . 25
9.3 Ordering and synchronization . . . 26
9.4 Errors and validation . . . 26
9.5 The frame loop . . . 26

**10. Project setup** **29**

10.1 Requirements . . . 29
10.2 Folder layout . . . 29
10.3 HTML page used by every example . . . 29
10.4 Shared helper: common.js . . . 29

**11. Example 01: Hello GPU: adapter, device, limits** **31**

**12. Example 02: Clear the screen (your first render pass)** **32**

**13. Example 03: Your first triangle** **33**

**14. Example 04: Vertex buffers and index buffers** **34**

**15. Example 05: Uniform buffers and a render loop** **36**

**16. Example 06: Textures and samplers** **39**

**17. Example 07: A rotating 3D cube with depth testing** **42**

**18. Example 08: Compute shader: process a million numbers on the GPU** **46**

**19. Example 09: GPU particle system: compute + instanced rendering** **48**

**20. Debugging, pitfalls and best practices** **52**

20.1 When nothing shows up . . . . . 52

20.2 Debugging tools . . . . . 52

20.3 Handling window resize and high-DPI screens . . . . . 52

20.4 Loading a real image as a texture . . . . . 53

20.5 Performance habits . . . . . 53

20.6 Cheat sheet . . . . . 54

**21. Where to go next** **55**

21.1 Good references . . . . . 55

21.2 Glossary . . . . . 56

# Part I
# Theory

The concepts behind WebGPU: the hardware, the architecture, the shading language and the two kinds of GPU work.

# 1. Introduction: what is WebGPU?

**In one sentence:** WebGPU is a modern web API that gives JavaScript direct, low-overhead access to the GPU for both **graphics (drawing)** and **general-purpose computation (compute)**.

## 1.1 What it is

WebGPU is a W3C standard exposed in the browser through `navigator.gpu`. Underneath, the browser translates your calls into the platform's native modern graphics API: **Vulkan** on Linux/Android, **Metal** on Apple devices and **Direct3D 12** on Windows. You write JavaScript for the setup and a new shading language called **WGSL** (WebGPU Shading Language) for the programs that run on the GPU.

## 1.2 Why was it created?

WebGL (2011) is a thin wrapper over OpenGL ES, an API design from the early 2000s. It is built around a giant global state machine: you flip switches (`gl.bindBuffer`, `gl.enable`, ...) and the driver figures out what to do at draw time. That caused three long-term problems:

- **High CPU overhead.** Validation and state translation happen on every draw call, which limits how many objects a scene can contain.
- **No compute.** WebGL has no general compute shaders. People abused fragment shaders and textures to do math, which is awkward and slow.
- **Mismatch with modern GPUs and drivers.** Vulkan, Metal and D3D12 use explicit objects (pipelines, command buffers). WebGL could not expose that model.

WebGPU adopts the explicit model: you build immutable objects up front (pipelines, bind groups), record commands into a command buffer, and submit the batch. Most validation happens when you create objects, not on every draw.

## 1.3 WebGL vs WebGPU at a glance

| Topic | WebGL / WebGL2 | WebGPU |
|---|---|---|
| Design basis | OpenGL ES (global state machine) | Vulkan / Metal / D3D12 style (explicit objects) |
| Shader language | GLSL | WGSL |
| Compute shaders | No | Yes, first-class |
| Storage buffers | No (textures as workaround) | Yes (large read/write buffers) |
| Pipeline state | Set piece by piece before each draw | Baked into an immutable pipeline object |
| Submitting work | Immediate calls, many small state changes | Record commands, then one `queue.submit()` |
| Resource binding | Bind to global slots | Bind groups (sets of resources) with explicit layouts |
| Errors | `gl.getError()` | Validation, error scopes, `uncapturederror` event |
| Async style | Mostly synchronous | Promises for device creation, buffer mapping, pipeline creation |

## 1.4 Browser support and requirements

WebGPU has shipped in Chrome and Edge (version 113 and later on desktop), and Firefox (from version 141, starting on Windows) and Safari 26 have shipped it as well, so it now covers all major engines. Exact platform coverage keeps changing, so check the current status on caniuse.com or MDN before you promise support to users. Two practical rules:

- The page must be a **secure context**: `https://` or `http://localhost`. Opening a file via `file://` is not reliable.
- Always **feature-detect**: `if (!navigator.gpu)` and also check that `requestAdapter()` does not return `null` (blocklisted driver, remote desktop, hardware acceleration disabled). Keep a WebGL2 or Canvas fallback for older devices.

## 1.5 What can you build with it?

- 3D games and visualizations with many more objects than WebGL allows
- GPU-accelerated machine learning in the browser (libraries such as ONNX Runtime Web, Transformers.js and WebLLM use it)
- Physics, particle systems, fluid and cloth simulation using compute shaders
- Video and image processing, scientific plotting, simulations with millions of points

> **Engines vs raw API**
>
> Libraries like Three.js and Babylon.js already include WebGPU renderers. You do not need raw WebGPU to ship a 3D app, but learning it teaches you what those engines do, and it is essential for custom compute or rendering work.

> **Key takeaways**
>
> - WebGPU = modern, explicit, low-overhead GPU access from JavaScript; shaders are written in WGSL.
> - It adds compute shaders and storage buffers, which WebGL lacks.
> - Use HTTPS or localhost, and feature-detect with a fallback.

# 2. GPU fundamentals

**In one sentence:** a GPU is a processor built to run the same small program on thousands of data items at the same time.

## 2.1 CPU vs GPU

| | CPU | GPU |
|---|---|---|
| Core count | A few to a few dozen powerful cores | Hundreds to thousands of simpler cores |
| Optimised for | Low latency, branching, general logic | High throughput on parallel data |
| Typical job | Game logic, file IO, UI, JavaScript | Shading millions of pixels, matrix math, simulation |
| Memory | System RAM (large, flexible) | VRAM (very high bandwidth); on integrated GPUs it is shared RAM |

A 1920x1080 frame has about two million pixels. Each pixel needs the same kind of calculation (lighting, texture lookup, ...). That is a perfect fit for a GPU: write the calculation once, run it two million times in parallel.

## 2.2 Threads, groups and lock-step execution

The GPU runs your shader as a huge number of small **invocations** (you can think of them as threads). The hardware groups them (NVIDIA calls a group a warp, AMD a wavefront) and executes each group in lock-step: all invocations in the group execute the same instruction at the same moment, on different data. Two consequences matter to you:

- **Divergence is costly.** If half the invocations in a group take an `if` branch and half take the `else`, the hardware runs both paths one after another. Keep branches simple when possible.
- **Invocations are independent.** You cannot assume an order, and invocations cannot freely read each other's results (except inside a compute workgroup, see chapter 8).

## 2.3 Memory and why copies matter

CPU and GPU usually have separate memory spaces. Data must be **uploaded** before the GPU can use it, and results must be **read back** to be seen by JavaScript. Both directions cost time, and readback also forces you to wait for the GPU. The golden rule of GPU programming: **move data once, compute on it many times, and avoid reading back every frame.**

## 2.4 Two kinds of GPU work

- **Rendering:** turn geometry (triangles) into pixels. Uses a fixed pipeline with two programmable stages (vertex and fragment shader).
- **Compute:** run any program over any data, with no triangles or pixels involved. Used for simulations, ML, image processing.

## 2.5 The GPU runs on its own timeline

When you call `queue.submit()`, JavaScript continues immediately; the GPU executes the work later, in parallel with your code. That is why anything that needs a GPU result (like reading a buffer) is **asynchronous** and returns a Promise.

**Key takeaways**

- A GPU runs one program on many data items at once; throughput matters more than per-thread speed.
- CPU and GPU memory are separate: upload once, avoid per-frame readback.
- GPU work is asynchronous: JavaScript records and submits, the GPU executes later.

# 3. WebGPU architecture

**In one sentence:** you ask the browser for a GPU adapter, open a device on it, create resources and pipelines from the device, record commands, and submit them to the device's queue.

## 3.1 The software stack

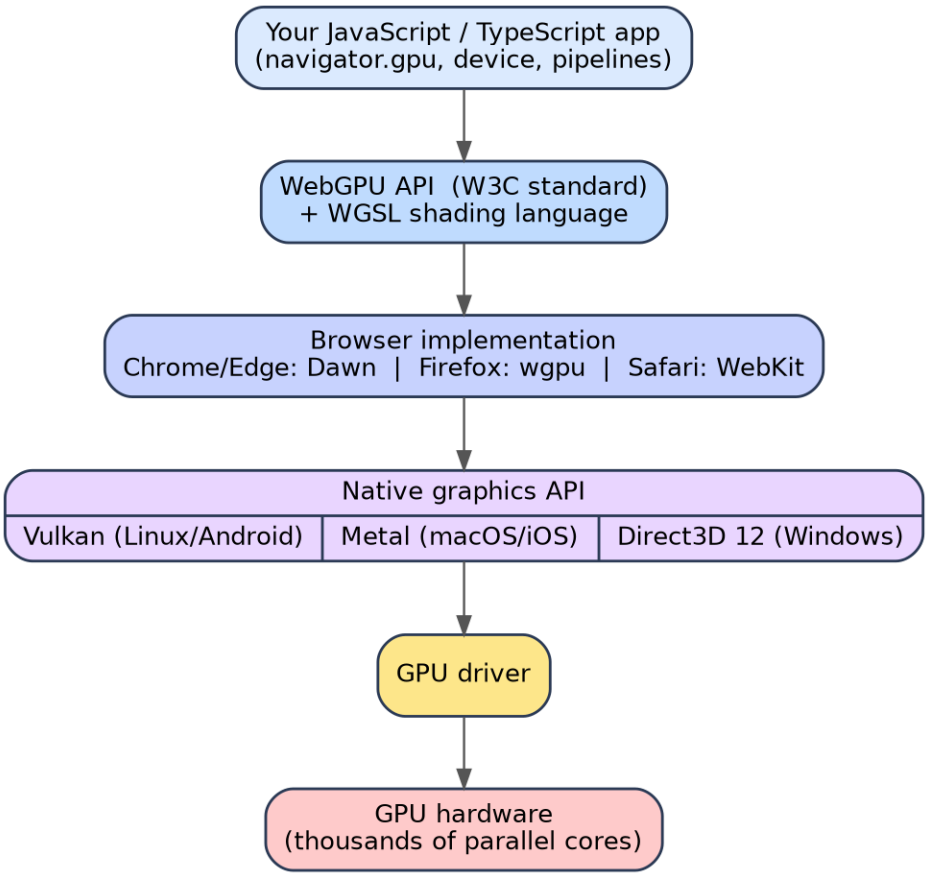



*Figure 1: Your JavaScript sits on top of the WebGPU API; the browser maps it to the native graphics API.*

Chrome and Edge use a C++ implementation called **Dawn**, Firefox uses the Rust-based **wgpu**, and Safari uses WebKit's own implementation on top of Metal. Behavior is standardized, but small differences (supported features, performance) exist, so test on more than one browser.

## 3.2 The object model

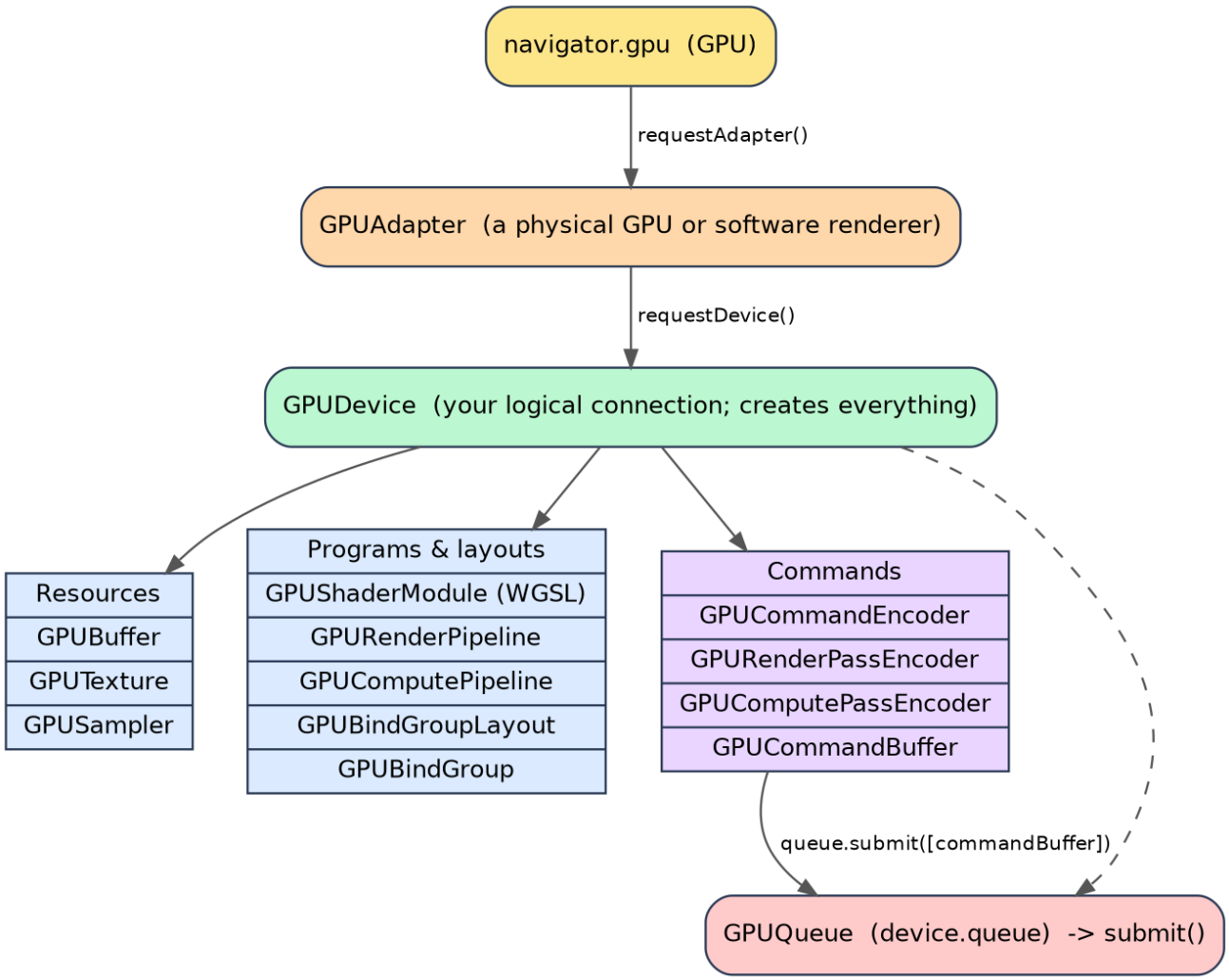



*Figure 2: Everything is created from the device. Commands are recorded in encoders and executed through the queue.*

## GPU and Adapter

`navigator.gpu` is the entry point. `await navigator.gpu.requestAdapter()` returns a **GPUAdapter**, which represents one physical GPU (or a software fallback). It tells you what hardware can do: `adapter.features` (optional capabilities such as `shader-f16`, `timestamp-query`) and `adapter.limits` (numbers such as maximum texture size). You can pass `{ powerPreference: "high-performance" }` on laptops with two GPUs.

## Device

`await adapter.requestDevice()` returns a **GPUDevice**: your logical connection to the GPU. It is the factory for everything: buffers, textures, pipelines, bind groups, command encoders. If you need optional features or higher limits, you must request them here: `requestDevice({ requiredFeatures: [...], requiredLimits: {...} })`. Anything not requested is not available, even if the adapter supports it.

## Queue

`device.queue` is where work is submitted. It offers `submit()`, and shortcuts for uploads: `writeBuffer()`, `writeTexture()`, `copyExternalImageToTexture()`.

## Device loss

Drivers can crash or the GPU can be reset. The promise `device.lost` resolves with a reason when that happens. A robust app listens for it and rebuilds its device and resources.

## 3.3 Features and limits

Limits have conservative defaults that every WebGPU device must satisfy. The important defaults:

| Limit | Default minimum | Meaning |
|---|---|---|
| maxTextureDimension2D | 8192 | Largest width or height of a 2D texture |
| maxBindGroups | 4 | Number of `@group(n)` sets a pipeline can use |
| maxBufferSize | 268435456 (256 MiB) | Largest single buffer you can create |
| maxStorageBufferBindingSize | 134217728 (128 MiB) | Largest storage buffer range a shader can bind |
| maxUniformBufferBindingSize | 65536 (64 KiB) | Largest uniform buffer range a shader can bind |
| maxComputeInvocationsPerWorkgroup | 256 | x * y * z of `@workgroup_size` must not exceed this |
| maxComputeWorkgroupSizeX / Y / Z | 256 / 256 / 64 | Per-axis workgroup size |
| maxComputeWorkgroupsPerDimension | 65535 | Per-axis count in `dispatchWorkgroups` |
| maxVertexBuffers | 8 | Vertex buffers bound at once |

**Key takeaways**

- adapter = physical GPU (query it); device = your logical handle (create things with it); queue = submit work.
- Optional features and bigger limits must be requested in `requestDevice`.
- Handle `device.lost` in real apps.

# 4. Resources: buffers, textures, samplers

**In one sentence:** resources are the GPU-side memory objects your shaders read and write; you declare their **usage flags** up front.

## 4.1 Buffers

A **GPUBuffer** is a block of raw bytes in GPU memory. You choose its `size` (bytes) and `usage` flags when creating it; the flags tell the browser how the buffer will be used so it can place and validate it properly. Flags can be combined with `|`.

| Usage flag | Meaning |
|---|---|
| VERTEX | Can be bound as a vertex buffer (per-vertex data) |
| INDEX | Can be bound as an index buffer |
| UNIFORM | Small, read-only data for shaders (matrices, time, settings) |
| STORAGE | Large buffers shaders can read, and optionally write (`read_write`) |
| COPY_DST | Can be the destination of copies/uploads (needed by `queue.writeBuffer`) |
| COPY_SRC | Can be the source of a copy |
| MAP_READ | CPU can map it and read bytes (only combinable with COPY_DST) |
| MAP_WRITE | CPU can map it and write bytes (only combinable with COPY_SRC) |
| INDIRECT | Holds draw/dispatch arguments generated on the GPU |

### Getting data in

- `device.queue.writeBuffer(buffer, offset, typedArray)`: the simplest way. Requires `COPY_DST`. Offset and size must be multiples of 4 bytes.
- `mappedAtCreation: true`: create the buffer already mapped, fill it with `getMappedRange()`, then `unmap()`. Good for static data.
- `mapAsync` on a `MAP_WRITE` staging buffer plus a copy: for large or frequent uploads.

### Getting data out

Shader results live in GPU memory. To read them in JavaScript you copy into a buffer with `MAP_READ | COPY_DST` using `encoder.copyBufferToBuffer`, submit, then `await buffer.mapAsync(GPUMapMode.READ)`, read via `getMappedRange()` and `unmap()`.

## 4.2 Textures

A **GPUTexture** is a 1D, 2D or 3D grid of texels with a **format**, optional mip levels and array layers. Textures hold images, render targets (what you draw into), depth buffers, and any gridded data. Like buffers they carry usage flags.

| Texture usage | Meaning |
|---|---|
| TEXTURE_BINDING | Shaders can sample it (`texture_2d<f32>`) |
| STORAGE_BINDING | Shaders can read/write texels directly (compute) |
| RENDER_ATTACHMENT | Can be drawn into (color or depth attachment) |
| COPY_SRC / COPY_DST | Source / destination of copies and uploads |

| Format | Typical use |
|---|---|
| rgba8unorm | Regular 8-bit-per-channel images; values 0..255 read as 0.0..1.0 in the shader |
| bgra8unorm | Often the canvas format on some platforms; always use `navigator.gpu.getPreferredCanvasFormat()` |
| rgba16float / rgba32float | High-dynamic-range data (float32 filtering may need an optional feature) |
| r32float | Single-channel float data for compute |
| depth24plus / depth32float | Depth buffers for 3D rendering |

You rarely use a texture directly; you create a **view** with `texture.createView()`, which describes which part (mips, layers, aspect) a shader or attachment sees.

## 4.3 Samplers

A **GPUSampler** describes how a shader reads a texture: filtering (`nearest` = crisp, `linear` = smooth), address mode (`clamp-to-edge`, `repeat`, `mirror-repeat`) and optionally anisotropy. Keeping this separate from the texture lets you reuse one texture with different sampling styles.

## 4.4 The canvas

To show pixels on screen you get a canvas context: `canvas.getContext("webgpu")`, then `context.configure({ device, format, alphaMode })`. Each frame, `context.getCurrentTexture()` returns the texture you should render into. **It is only valid for the current frame**, so request it every frame instead of caching it.

> **Key takeaways**
>
> - Always set correct `usage` flags: most beginner errors are missing flags.
> - `writeBuffer` for uploads; copy to a `MAP_READ` buffer plus `mapAsync` for readback.
> - Textures are used through views; samplers are separate objects; canvas texture must be fetched each frame.

# 5. WGSL: the WebGPU Shading Language

**In one sentence:** WGSL is a small, strongly typed, Rust-flavored language whose programs run on the GPU as vertex, fragment or compute shaders.

## 5.1 First look

**wgsl-overview.wgsl**

```
// A tiny complete example
struct Material { color: vec4f, shininess: f32 };

@group(0) @binding(0) var<uniform> material: Material;

fn brighten(c: vec3f, amount: f32) -> vec3f {
  return clamp(c * amount, vec3f(0.0), vec3f(1.0));
}

@fragment
fn fs() -> @location(0) vec4f {
  let rgb = brighten(material.color.rgb, 1.2);
  return vec4f(rgb, material.color.a);
}
```

- Functions: `fn name(arg: type) -> type { ... }`
- Declarations: `let` (immutable value), `var` (mutable), `const` (compile-time constant)
- Types come after names, like Rust/TypeScript: `let x: f32 = 1.0;` (usually inferred)
- Statements end with `;`. Structs list members separated by commas.
- Attributes start with `@` and attach meaning: `@vertex`, `@location(0)`, `@builtin(position)`, `@group(0)`, `@binding(1)`.

## 5.2 Types

| Type | Meaning | Short alias / notes |
|---|---|---|
| bool | true / false | |
| i32, u32 | 32-bit signed / unsigned integers | `vec3u`, `vec2i` ... |
| f32 | 32-bit float (the workhorse) | `vec2f`, `vec3f`, `vec4f` |
| f16 | 16-bit float | Requires the `shader-f16` feature and `enable f16;` |
| vecN<T> | N-component vector | `vec3<f32>` = `vec3f`; swizzles: `v.xyz`, `c.rgb`, `v.xx` |
| matCxR<T> | C columns, R rows (column-major) | `mat4x4f`, `mat3x3f` |
| array<T, N> | Fixed-size array | `array<T>` (runtime-sized) allowed as the last storage buffer member |
| struct | Named group of members | Used for IO between stages and for buffer layout |
| texture_2d<f32>, sampler | Texture and sampler handles | Used with `textureSample`, `textureLoad` |

WGSL has **no implicit conversions between concrete types**. `let a: f32 = 2;` works because the literal `2` is an abstract integer that is allowed to become `f32`, but `let b = u32_value + 1.0;` is an error. Convert explicitly: `f32(i)`, `u32(x)`, `vec3f(v)`.

## 5.3 Control flow

wgsl-control.wgsl

```
for (var i = 0u; i < 10u; i++) {
  if (i == 3u) { continue; }
  sum += f32(i);
}
// Also: while, loop { ... break; }, switch, select(falseValue, trueValue, condition)
let c = select(0.0, 1.0, x > 0.5);   // branchless "ternary"
```

## 5.4 Address spaces (where variables live)

| Address space | Declared as | Who can use it | Access |
|---|---|---|---|
| function | `var x: f32` inside a function | One invocation | read_write |
| private | `var<private> x: f32` at module level | One invocation | read_write |
| workgroup | `var<workgroup> tile: array<f32, 64>` | All invocations in a compute workgroup | read_write (fast shared memory) |
| uniform | `var<uniform> u: Uniforms` | All invocations | read-only, small (<= 64 KiB default) |
| storage | `var<storage, read> / read_write` | All invocations | large buffers; read_write only in compute |
| handle | `var t: texture_2d<f32>` | All invocations | textures and samplers |

## 5.5 Entry points and attributes

A shader module can contain many functions; the ones the pipeline can start from are **entry points**, marked by a stage attribute.

| Attribute | Where | Meaning |
|---|---|---|
| @vertex | function | Vertex stage entry; must return `@builtin(position) vec4f` (directly or in a struct) |
| @fragment | function | Fragment stage entry; returns `@location(n)` colors |
| @compute @workgroup_size(x,y,z) | function | Compute entry; workgroup size is baked in |
| @location(n) | inputs/outputs | Numbered slots: vertex attributes in, fragment color targets out, and stage-to-stage values |
| @group(g) @binding(b) | module variables | Which bind-group slot supplies this resource |
| @builtin(name) | inputs/outputs | Hardware-provided values (table below) |

| Builtin | Stage | Type | Meaning |
|---|---|---|---|
| vertex_index | vertex in | u32 | Index of the current vertex (0..draw count-1) |
| instance_index | vertex in | u32 | Index of the current instance |
| position | vertex out / fragment in | vec4f | Clip-space position / window position of the pixel |
| front_facing | fragment in | bool | Is the triangle facing the camera |
| global_invocation_id | compute in | vec3u | Unique 3D id of the thread across the whole dispatch |
| local_invocation_id | compute in | vec3u | Thread id inside its workgroup |
| workgroup_id | compute in | vec3u | Which workgroup this thread belongs to |

| Builtin | Stage | Type | Meaning |
|---|---|---|---|
| num_workgroups | compute in | vec3u | Dispatch size in workgroups |
| local_invocation_index | compute in | u32 | Flattened thread index inside the workgroup |

## 5.6 Memory layout rules (very important for buffers)

When a struct lives in a **uniform** or **storage** buffer, JavaScript must write bytes in exactly the layout WGSL expects. Each type has an **alignment** (its start address must be a multiple of it) and a **size**:

| WGSL type | Alignment (bytes) | Size (bytes) |
|---|---|---|
| f32, i32, u32 | 4 | 4 |
| vec2<f32> | 8 | 8 |
| vec3<f32> | 16 | 12 (**the classic trap**: 4 bytes of padding follow) |
| vec4<f32> | 16 | 16 |
| mat3x3<f32> | 16 | 48 (three columns, each padded to 16) |
| mat4x4<f32> | 16 | 64 |
| struct | largest member alignment | end of last member, rounded up to the struct alignment |
| array<T, N> | alignment of T | N * roundUp(alignOf(T), sizeOf(T)); in uniform buffers the element stride must be a multiple of 16 |

**layout-example.wgsl + js**

```
struct Light {
  color: vec3f,     // offset 0,  size 12
  intensity: f32,   // offset 12 (fits in the vec3's padding!), size 4
  position: vec3f,  // offset 16, size 12
  radius: f32,      // offset 28
};                  // total size 32

// JS side
const data = new Float32Array([
  1, 0.9, 0.8,  2.5,     // color.rgb, intensity
  0, 10, 0,     5.0,     // position.xyz, radius
]);
```

> **Tip**
>
> Order members so that a vec3 is followed by a f32 (it fills the padding), or use vec4 everywhere. If something renders as garbage, suspect alignment first.

## 5.7 Useful built-in functions

Math: `abs, min, max, clamp, mix, step, smoothstep, sin, cos, pow, sqrt, floor, fract, length, normalize, dot, cross, reflect`. Textures: `textureSample, textureLoad, textureStore, textureDimensions`. Compute sync: `workgroupBarrier`. Atomics: `atomicAdd` etc. on `atomic<u32>` variables in storage or workgroup memory.

> **Key takeaways**
>
> - WGSL is strict: explicit types, explicit conversions.
> - Know the five address spaces; `workgroup` is the fast shared memory.
> - Respect alignment: vec3 has size 12 but alignment 16.

# 6. Pipelines and resource binding

**In one sentence:** a pipeline is an immutable object that bundles your shaders and all fixed-function settings; a bind group supplies the resources those shaders use.

## 6.1 Pipelines

Instead of setting state before every draw, you describe it once in a pipeline. The browser validates and compiles it at creation, so drawing later is cheap.

- **GPURenderPipeline**: vertex shader, fragment shader, vertex buffer layouts, primitive topology, culling, depth/stencil state, blending, color target formats, multisampling.
- **GPUComputePipeline**: one compute shader entry point.

Pipeline creation can be slow because shaders compile. Prefer `createRenderPipelineAsync` / `createComputePipelineAsync` for big shaders so you do not stall the main thread, and create pipelines at load time, not per frame.

## 6.2 Bind groups and layouts

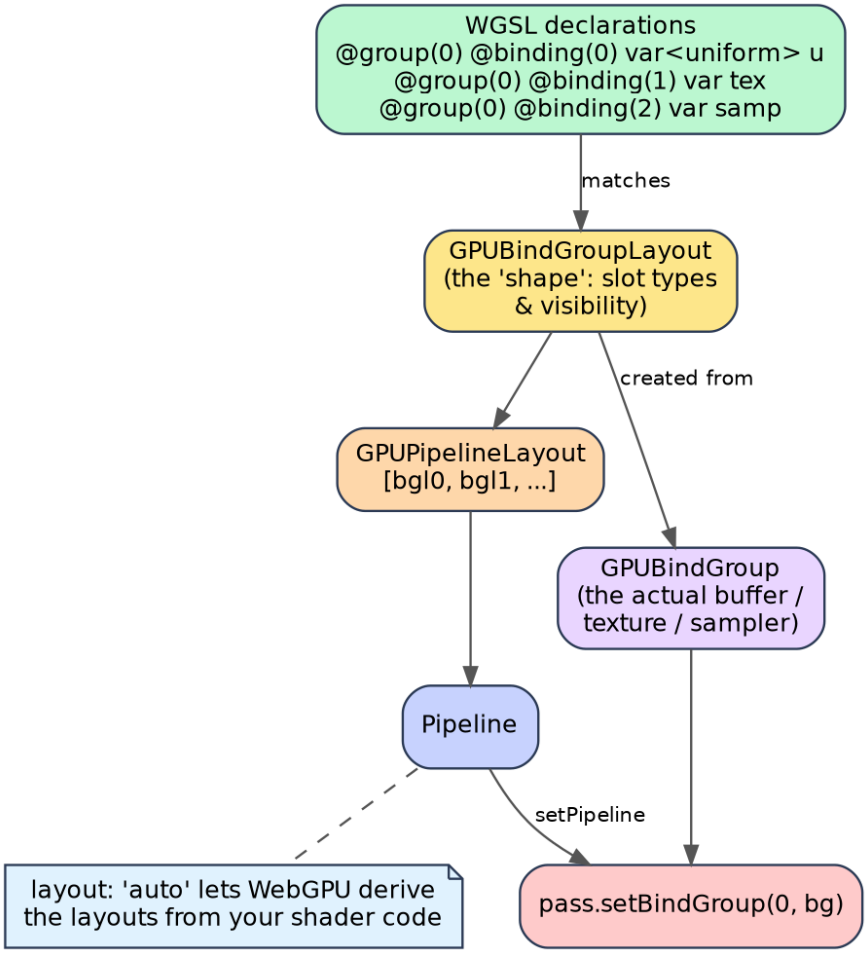



*Figure 3: WGSL declares slots; the layout describes their shape; the bind group fills them with real resources.*

In WGSL you declare resources with `@group(g) @binding(b)`. On the JavaScript side:

- A **GPUBindGroupLayout** says what kind of resource sits at each binding (uniform buffer, storage buffer, texture, sampler) and which shader stages can see it.
- A **GPUBindGroup** is an instance of a layout filled with actual buffers/textures/samplers. Bind groups are **immutable**: to change what is bound, make a new one (or change the buffer's

contents, which is the normal approach).

- A **GPUPipelineLayout** is a list of bind-group layouts, one per `@group` index.

With `layout: "auto"` the browser derives all layouts from your shader code, and you fetch them with `pipeline.getBindGroupLayout(index)`. That is perfect for learning and small apps. For sharing one bind group across many pipelines, create explicit layouts with `device.createBindGroupLayout` and `device.createPipelineLayout`.

**Good habit**

Group resources by how often they change: per-frame data (camera) in group 0, per-material data in group 1, per-object data in group 2. Then you rebind only the groups that changed.

**Key takeaways**

- Pipelines are immutable and validated up front: create them once.
- `@group/@binding` in WGSL must match the layout and the bind group entries.
- `layout: "auto"` + `getBindGroupLayout(i)` is the easiest start.

# 7. The render pipeline in depth

**In one sentence:** the GPU turns vertex data into pixels through a fixed sequence of stages, two of which (vertex and fragment shaders) are yours.

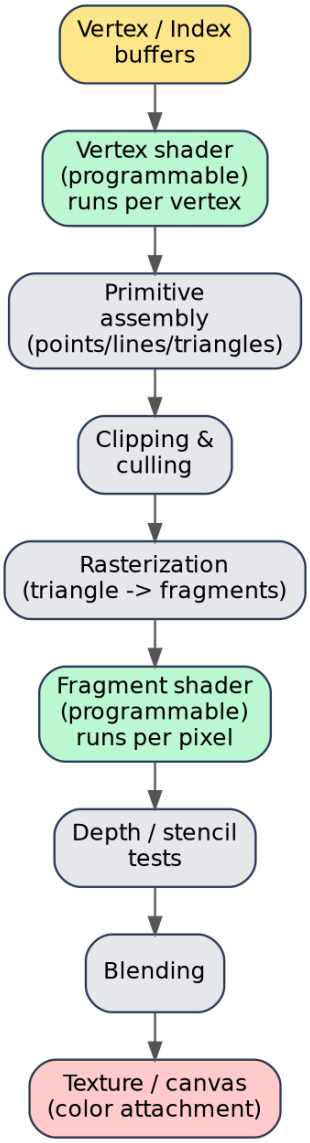



*Figure 4: Green stages are programmable by you; grey stages are fixed-function but configurable in the pipeline descriptor.*

## 7.1 Walking through the stages

- **Vertex fetch:** reads vertex (and optional index) buffers according to the `buffers` layout in the pipeline.
- **Vertex shader:** runs once per vertex. Its main job: output the vertex position in **clip space** (`@builtin(position)`). It can also output extra values (colors, UVs, normals) that get interpolated.
- **Primitive assembly:** groups vertices into primitives based on `topology`: `point-list`, `line-list`, `line-strip`, `triangle-list`, `triangle-strip`.
- **Clipping and culling:** primitives outside the view are cut away; with `cullMode: "back"` triangles facing away from the camera are discarded.
- **Rasterization:** each triangle is converted to the set of pixels (fragments) it covers. Per-vertex outputs are **interpolated** across the triangle.

- **Fragment shader:** runs once per fragment and returns a color (`@location(0)`).
- **Depth/stencil test:** keeps the fragment only if it is nearer than what is already stored in the depth buffer.
- **Blending and write:** the color is combined with what is already in the target (for transparency) and stored.

## 7.2 Coordinate systems

| Space | Range / convention |
|---|---|
| Clip space (vertex shader output) | x, y, z, w. After dividing by w you get normalized device coordinates. |
| NDC (normalized device coordinates) | x: -1 (left) to +1 (right); y: -1 (bottom) to +1 (top); **z: 0 (near) to 1 (far)**. Note z is 0..1, unlike OpenGL's -1..1. |
| Framebuffer / pixel coordinates | Origin at the **top-left**, y grows downward (`@builtin(position)` in the fragment shader). |
| Texture coordinates (UV) | (0,0) is the **top-left** of the texture, (1,1) bottom-right. |

## 7.3 Winding order and culling

A triangle's **front face** is the one whose vertices appear counter-clockwise (the default `frontFace: "ccw"`) on screen. With `cullMode: "back"`, clockwise triangles are skipped, which halves the work for closed meshes such as a cube. If your mesh disappears, try `cullMode: "none"` to check whether winding is the cause.

## 7.4 The MVP transform

To place a 3D model on screen you multiply each vertex by three matrices, usually combined into one on the CPU:

**MVP**

```
clipPosition = Projection * View * Model * vec4(position, 1)
```

- **Model:** moves/rotates/scales the object into the world.
- **View:** moves the world so the camera sits at the origin looking down -Z.
- **Projection:** applies perspective (far things look smaller) and maps depth to 0..1.

Matrices are **column-major** in WGSL, so a `Float32Array(16)` should list the first column, then the second, and so on. The translation lives in elements 12, 13, 14.

## 7.5 Depth buffer

Without depth testing, whichever triangle is drawn last wins. A depth texture (e.g. `depth24plus`) stores the nearest distance per pixel; with `depthCompare: "less"` the GPU keeps only closer fragments, so draw order stops mattering for opaque objects.

## 7.6 Render pass: attachments, load and store

A render pass draws into **attachments**: one or more color textures and optionally a depth/stencil texture. For each attachment:

- `loadOp: "clear"` fills with `clearValue` at the start; `"load"` keeps the existing content.
- `storeOp: "store"` keeps the result; `"discard"` throws it away (useful for temporary depth buffers).

## 7.7 Instancing

`pass.draw(vertexCount, instanceCount)` draws the same geometry many times; the vertex shader can read `@builtin(instance_index)` to look up per-instance data (position, color, transform) from a buffer. This is how you draw thousands of copies with one call (we use it in the particle example).

> **Key takeaways**
>
> - You write the vertex and fragment shaders; the GPU does the rest in between.
> - Clip-space depth is 0..1; texture origin is top-left; default front face is counter-clockwise.
> - Depth testing needs a depth texture and a `depthStencil` state in the pipeline.

# 8. The compute pipeline in depth

**In one sentence:** a compute shader runs a function over a grid of invocations organized into workgroups, reading and writing storage buffers.

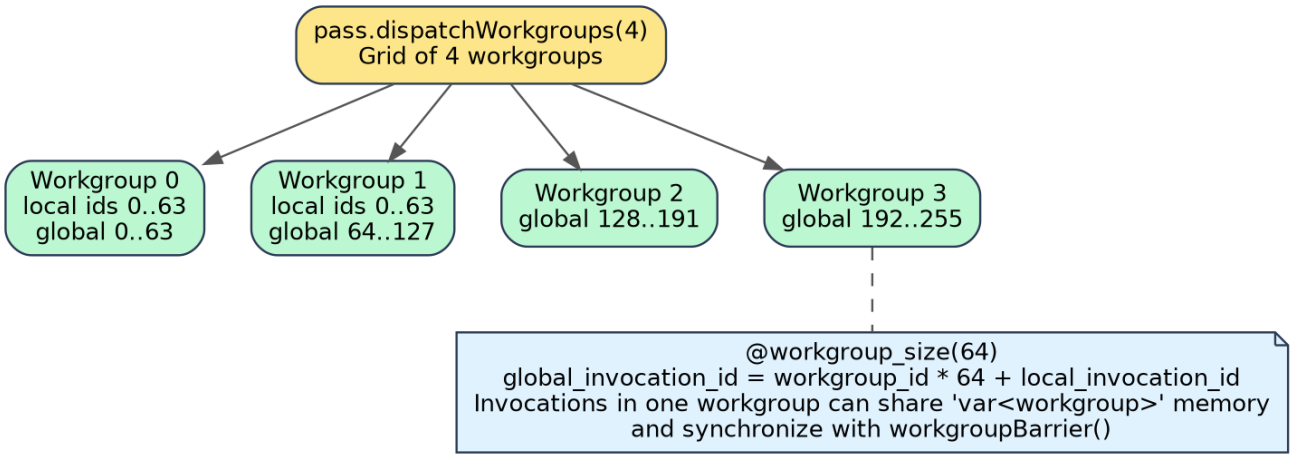



*Figure 5: A dispatch launches a grid of workgroups; each workgroup runs `@workgroup_size` invocations.*

## 8.1 Workgroups

You choose two numbers: the **workgroup size** inside the shader (`@workgroup_size(64)`) and the **number of workgroups** in JavaScript (`pass.dispatchWorkgroups(n)`). The total number of invocations is the product. Each invocation sees:

- `local_invocation_id`: its index inside its workgroup (0..63)
- `workgroup_id`: which workgroup it is in
- `global_invocation_id`: `workgroup_id * workgroup_size + local_invocation_id`, the unique index you usually use to pick which element to process

**dispatch math**

```
// For N = 1000 items with workgroup_size(64):
// workgroups = ceil(1000 / 64) = 16  -> 1024 invocations
// the last 24 invocations are "extra", so always guard:
if (id.x >= arrayLength(&data)) { return; }
```

## 8.2 Choosing a workgroup size

- 64 is a safe default for 1D work; 256 is the maximum on default limits; 8x8 (=64) is common for 2D image work.
- Use multiples of 32 (NVIDIA warp) and 64 (AMD wavefront) so the hardware is fully used.
- Never exceed `maxComputeInvocationsPerWorkgroup` (default 256) or per-axis size limits.

## 8.3 Workgroup memory and barriers

Invocations in the **same** workgroup can cooperate through `var<workgroup>` memory. After writing to it, call `workgroupBarrier()` so that every invocation sees everyone's writes before reading. This is the basis of reductions (sums), prefix scans, tiled matrix multiply and image blurs. There is **no** barrier across different workgroups inside one dispatch; for that, use separate dispatches.

## 8.4 Reading results

Compute output stays in storage buffers or storage textures. Either use it directly as input to the next pass (best, no copy) or read it back with the copy + `mapAsync` pattern from chapter 4. Readback stalls the pipeline, so avoid it per frame.

## 8.5 Race conditions and atomics

If many invocations write the same location without coordination, results are undefined. Either give each invocation its own output index, or use `atomic<u32>` / `atomic<i32>` variables with `atomicAdd`, `atomicMax`, etc. for counters and histograms.

**Key takeaways**

- Pick `@workgroup_size` in WGSL, pick the dispatch count in JS, guard against out-of-range ids.
- Same-workgroup invocations share `var<workgroup>` memory and synchronize with `workgroupBarrier()`.
- Keep data on the GPU between passes; read back only when you must.

# 9. Command encoding and data flow

**In one sentence:** you record GPU work into a command encoder (cheap, on the CPU), finish it into a command buffer, and submit it to the queue (the GPU runs it later).

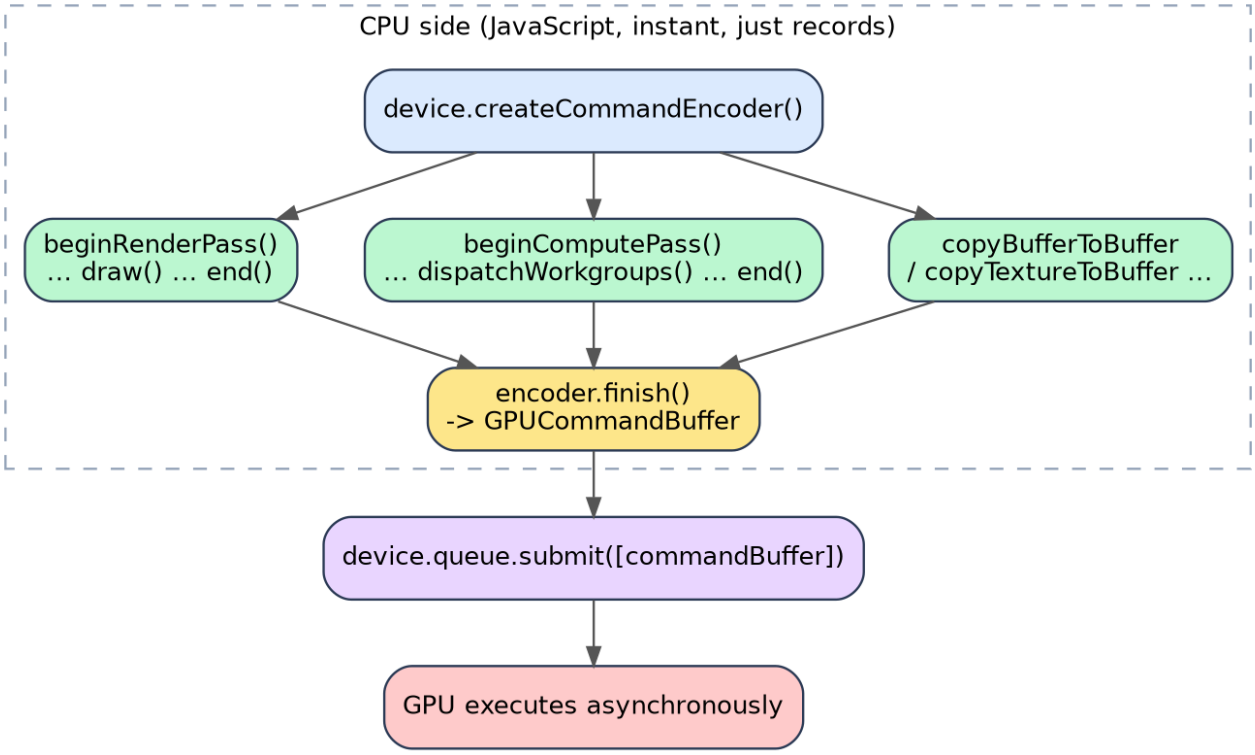



*Figure 6: Recording is instant and CPU-side. Nothing executes on the GPU until `queue.submit()`.*

## 9.1 Command encoder

`device.createCommandEncoder()` returns a recorder. On it you can: begin a render pass, begin a compute pass, or record copies (`copyBufferToBuffer`, `copyBufferToTexture`, `copyTextureToBuffer`, `copyTextureToTexture`, `clearBuffer`). A pass ends with `pass.end()`. `encoder.finish()` returns an immutable **GPUCommandBuffer**. One encoder can hold many passes, which are executed in order. Putting a compute pass and a render pass in the same encoder, as in the particle example, is typical.

## 9.2 Moving data

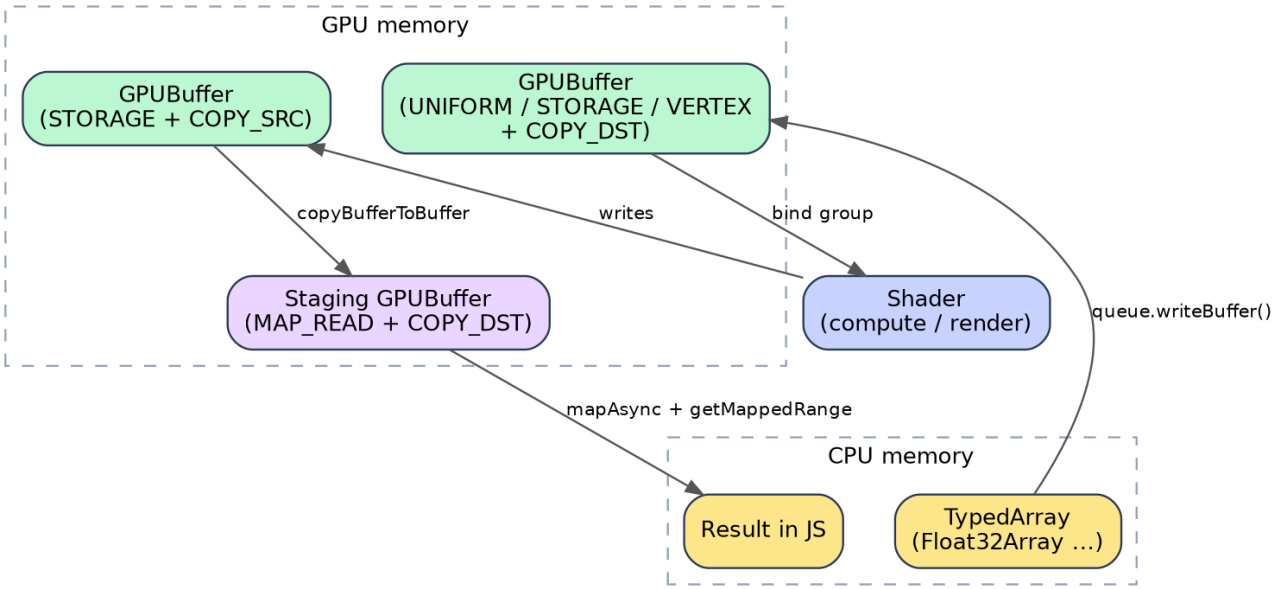



*Figure 7: Typical life of data: JS array, GPU buffer, shader, output buffer, staging buffer, JS array.*

| Method | When to use |
|---|---|
| `queue.writeBuffer / writeTexture` | Default choice for CPU to GPU uploads; WebGPU stages the copy for you |
| `mappedAtCreation` | Fill a new buffer immediately with static data |
| `copyBufferToBuffer` + `mapAsync` | GPU to CPU readback |
| `copyExternalImageToTexture` | Upload an `ImageBitmap`, canvas, or video frame into a texture |

## 9.3 Ordering and synchronization

Within a queue, operations happen in submission order, and `writeBuffer` calls take effect in order relative to `submit` calls. You do not need fences for normal rendering. The only time you wait is when you `await` a promise: `mapAsync`, `device.queue.onSubmittedWorkDone()`, or async pipeline creation.

## 9.4 Errors and validation

WebGPU validates aggressively. An invalid call does not throw in JavaScript; it creates an **error object** and reports the message asynchronously. Three tools help:

- `device.addEventListener("uncapturederror", e => console.error(e.error.message))` catches all unhandled errors (it is in our `common.js`).
- `device.pushErrorScope("validation")` … `await device.popErrorScope()` wraps a region of code and returns the first error inside.
- `await shaderModule.getCompilationInfo()` lists WGSL compile errors with line numbers.

Browser developer tools also print readable WebGPU validation messages. **Always read the first error**: later ones are usually consequences of it.

## 9.5 The frame loop

**frame loop pattern**

```
function frame(timeMs) {
  update(timeMs);                    // 1. write new uniform/storage data
  const encoder = device.createCommandEncoder();
  // 2. record compute and/or render passes
  device.queue.submit([encoder.finish()]);   // 3. one submit per frame
  requestAnimationFrame(frame);      // 4. schedule next frame (vsync-paced)
}
requestAnimationFrame(frame);
```

### Key takeaways

- Record with an encoder, `finish()`, then `submit()`: one submit per frame is ideal.
- Choose the right data-movement tool; avoid per-frame readback.
- Errors are asynchronous: listen for `uncapturederror` and read the first message.

# Part II
# Hands-on JavaScript

Nine small programs, each introducing one new idea. Type them yourself or run the ready-made files from the companion zip.

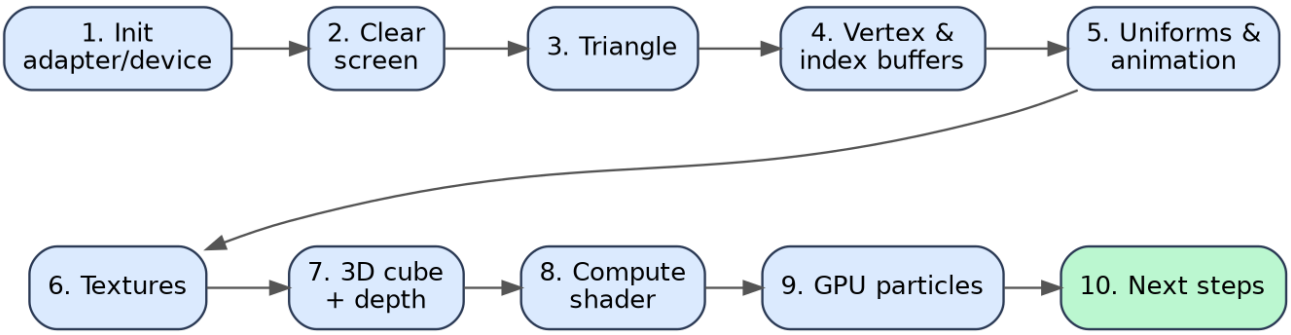



*Figure 8: The learning path of Part II. Each step reuses everything from the previous one.*

# 10. Project setup

## 10.1 Requirements

- A browser with WebGPU: current Chrome, Edge, Firefox or Safari (see chapter 1.4).
- A code editor (VS Code is fine) and a way to serve static files over `localhost`.
- Node.js is optional; it only provides the tiny server: `npx serve .` or `npx http-server .`. Python works too: `python3 -m http.server 8000`.

## 10.2 Folder layout

**layout**

```
webgpu-course/
  common.js          <- shared helper (init, logging)
  math.js            <- tiny matrix helpers (used from example 07)
  01-hello-gpu.html  + 01-hello-gpu.js
  02-clear-screen.html + 02-clear-screen.js
  ...
  09-gpu-particles.html + 09-gpu-particles.js
```

## 10.3 HTML page used by every example

**example.html**

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8" />
  <title>WebGPU example</title>
  <style>
    body { margin: 0; background: #111; color: #ddd; font-family: monospace; }
    canvas { display: block; margin: 20px auto; background: #000; }
    #log { margin: 20px; white-space: pre-wrap; }
  </style>
</head>
<body>
  <canvas id="gpu-canvas" width="640" height="480"></canvas>
  <pre id="log"></pre>
  <script type="module" src="./main.js"></script>
</body>
</html>
```

The page only provides a canvas, a `<pre>` for text output, and loads the JavaScript as an ES module (`type="module"` gives you `import` and top-level `await`).

## 10.4 Shared helper: common.js

Every example starts by calling `initGPU`. It performs the steps from chapter 3 and installs the error listeners from chapter 9.

**common.js**

```
// common.js - shared helpers used by every example
export async function initGPU(canvas) {
  if (!navigator.gpu) {
    throw new Error("WebGPU is not supported in this browser.");
  }
  const adapter = await navigator.gpu.requestAdapter();
  if (!adapter) {
    throw new Error("No suitable GPU adapter found.");
  }
  const device = await adapter.requestDevice();

  // Helpful diagnostics: you will thank yourself later.
  device.lost.then((info) => {
    console.error(`Device lost (${info.reason}): ${info.message}`);
  });
  device.addEventListener("uncapturederror", (event) => {
    console.error("WebGPU error:", event.error.message);
  });

  let context = null;
  let format = null;
  if (canvas) {
    context = canvas.getContext("webgpu");
    format = navigator.gpu.getPreferredCanvasFormat();
    context.configure({ device, format, alphaMode: "opaque" });
  }
  return { adapter, device, context, format };
}

// Prints to the console AND to the <pre id="log"> element on the page.
export function log(...args) {
  console.log(...args);
  const el = document.getElementById("log");
  if (el) el.textContent += args.join(" ") + "\n";
}
```

- `requestAdapter()` may return `null`: we turn that into a clear error message.
- `getPreferredCanvasFormat()` returns the format that is fastest for the user's screen (`bgra8unorm` or `rgba8unorm`). We must use the same format for the pipeline's color target.
- `alphaMode: "opaque"` means the canvas ignores alpha when compositing with the page.

# 11. Example 01: Hello GPU: adapter, device, limits

**Goal: confirm WebGPU works and learn what your GPU can do.**

**01-hello-gpu.js**

```
import { initGPU, log } from "./common.js";

const { adapter, device } = await initGPU();   // no canvas needed

const info = adapter.info ?? {};
log("Vendor:      ", info.vendor ?? "n/a");
log("Architecture:", info.architecture ?? "n/a");
log("Description: ", info.description ?? "n/a");
log("Features:    ", [...adapter.features].join(", ") || "(none)");

log("maxTextureDimension2D:", device.limits.maxTextureDimension2D);
log("maxBufferSize:", device.limits.maxBufferSize);
log("maxStorageBufferBindingSize:", device.limits.maxStorageBufferBindingSize);
log("maxComputeInvocationsPerWorkgroup:",
    device.limits.maxComputeInvocationsPerWorkgroup);
log("maxComputeWorkgroupSizeX:", device.limits.maxComputeWorkgroupSizeX);
```

## What to notice

- `adapter.info` (vendor, architecture) is available in current browsers; the code falls back to `n/a` if missing.
- `device.limits` shows the real limits of your machine (compare with chapter 3's table).
- Expected result: a list of text lines on the page. If you see an error, open the console: the likely causes are an unsupported browser, a blocked GPU, or a non-secure page.

# 12. Example 02: Clear the screen (your first render pass)

**Goal: the smallest render pass. This introduces the command encoder pattern you will use forever.**

02-clear-screen.js

```
import { initGPU } from "./common.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context } = await initGPU(canvas);

function render() {
  // 1. Record commands
  const encoder = device.createCommandEncoder({ label: "clear-encoder" });

  const pass = encoder.beginRenderPass({
    colorAttachments: [{
      view: context.getCurrentTexture().createView(), // where to draw
      clearValue: { r: 0.05, g: 0.15, b: 0.35, a: 1.0 },
      loadOp: "clear",   // start by filling with clearValue
      storeOp: "store",  // keep the result when the pass ends
    }],
  });
  pass.end();            // no draw calls: we only wanted the clear

  // 2. Finish recording and submit to the GPU
  device.queue.submit([encoder.finish()]);
}

render();
```

## What to notice

- `getCurrentTexture().createView()` is the screen for this frame.
- `loadOp: "clear"` + `clearValue` fills the target with a color. Change the numbers (0..1) and reload.
- We never call `draw`; a pass with only a clear is perfectly valid.
- Expected result: a solid dark-blue canvas.

# 13. Example 03: Your first triangle

**Goal: draw a triangle with no buffers at all, to see the shader and pipeline machinery in isolation.**

**03-triangle.js**

```
import { initGPU } from "./common.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context, format } = await initGPU(canvas);

const shader = device.createShaderModule({
  label: "triangle shader",
  code: /* wgsl */ `
    @vertex
    fn vs(@builtin(vertex_index) i: u32) -> @builtin(position) vec4f {
      var positions = array<vec2f, 3>(
        vec2f( 0.0,  0.6),   // top
        vec2f(-0.6, -0.6),   // bottom-left
        vec2f( 0.6, -0.6)    // bottom-right
      );
      return vec4f(positions[i], 0.0, 1.0);
    }

    @fragment
    fn fs() -> @location(0) vec4f {
      return vec4f(1.0, 0.45, 0.15, 1.0);   // orange (r, g, b, a)
    }
  `,
});

const pipeline = device.createRenderPipeline({
  label: "triangle pipeline",
  layout: "auto",
  vertex:   { module: shader, entryPoint: "vs" },
  fragment: { module: shader, entryPoint: "fs", targets: [{ format }] },
  primitive: { topology: "triangle-list" },
});

const encoder = device.createCommandEncoder();
const pass = encoder.beginRenderPass({
  colorAttachments: [{
    view: context.getCurrentTexture().createView(),
    clearValue: { r: 0.08, g: 0.08, b: 0.12, a: 1 },
    loadOp: "clear",
    storeOp: "store",
  }],
});
pass.setPipeline(pipeline);
pass.draw(3);            // 3 vertices -> vertex shader runs 3 times
```

```
pass.end();
device.queue.submit([encoder.finish()]);
```

## What to notice

- The vertex shader receives `@builtin(vertex_index)` (0, 1, 2) and picks one of three hard-coded positions. Positions are already in clip space.
- The fragment shader returns one constant orange color for every pixel of the triangle.
- `layout: "auto"` because this shader uses no resources.
- The pipeline's color target `format` must equal the canvas format.
- Expected result: an orange triangle on a dark background.

# 14. Example 04: Vertex buffers and index buffers

**Goal: feed real data from JavaScript. Introduces vertex buffers, vertex attribute layout, interleaving, index buffers and interpolation.**

**04-vertex-index-buffers.js**

```
import { initGPU } from "./common.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context, format } = await initGPU(canvas);

// x, y, r, g, b  (interleaved: 5 floats = 20 bytes per vertex)
const vertices = new Float32Array([
  -0.5, -0.5,   1, 0, 0,   // 0 bottom-left  red
   0.5, -0.5,   0, 1, 0,   // 1 bottom-right green
   0.5,  0.5,   0, 0, 1,   // 2 top-right    blue
  -0.5,  0.5,   1, 1, 0,   // 3 top-left     yellow
]);
// Two triangles that share vertices (counter-clockwise winding)
const indices = new Uint16Array([0, 1, 2,   0, 2, 3]);

const vertexBuffer = device.createBuffer({
  label: "vertex buffer",
  size: vertices.byteLength,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(vertexBuffer, 0, vertices);

const indexBuffer = device.createBuffer({
  label: "index buffer",
  size: indices.byteLength,            // 12 bytes: multiple of 4, good
  usage: GPUBufferUsage.INDEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(indexBuffer, 0, indices);

const shader = device.createShaderModule({
  code: /* wgsl */ `
    struct VSOut {
      @builtin(position) pos: vec4f,
      @location(0) color: vec3f,
    };

    @vertex
    fn vs(@location(0) position: vec2f,
          @location(1) color: vec3f) -> VSOut {
      var output: VSOut;
      output.pos = vec4f(position, 0.0, 1.0);
      output.color = color;
      return output;
    }
```

```
    @fragment
    fn fs(input: VSOut) -> @location(0) vec4f {
      return vec4f(input.color, 1.0);   // color is interpolated per pixel
    }
  `,
});

const pipeline = device.createRenderPipeline({
  layout: "auto",
  vertex: {
    module: shader,
    entryPoint: "vs",
    buffers: [{
      arrayStride: 5 * 4,                       // bytes between vertices
      attributes: [
        { shaderLocation: 0, offset: 0,     format: "float32x2" }, // position
        { shaderLocation: 1, offset: 2 * 4, format: "float32x3" }, // color
      ],
    }],
  },
  fragment: { module: shader, entryPoint: "fs", targets: [{ format }] },
  primitive: { topology: "triangle-list" },
});

const encoder = device.createCommandEncoder();
const pass = encoder.beginRenderPass({
  colorAttachments: [{
    view: context.getCurrentTexture().createView(),
    clearValue: { r: 0.08, g: 0.08, b: 0.12, a: 1 },
    loadOp: "clear", storeOp: "store",
  }],
});
pass.setPipeline(pipeline);
pass.setVertexBuffer(0, vertexBuffer);
pass.setIndexBuffer(indexBuffer, "uint16");
pass.drawIndexed(6);
pass.end();
device.queue.submit([encoder.finish()]);
```

## What to notice

- The vertex array interleaves position and color: 5 floats per vertex, so `arrayStride` = 20 bytes.
- `attributes` map bytes to shader inputs: `shaderLocation` matches `@location(n)` in WGSL, `offset` is the byte offset, `format` the data type.
- Four vertices + six indices = a quad made of two triangles; indices avoid duplicating shared corners.
- The color is output from the vertex shader and arrives **interpolated** in the fragment shader: you get a smooth gradient for free.
- `writeBuffer` requires a size multiple of 4: six `Uint16` = 12 bytes works; five would fail.
- Expected result: a square with red, green, blue and yellow corners blending into each other.

# 15. Example 05: Uniform buffers and a render loop

**Goal: change data every frame. Introduces uniform buffers, bind groups, `requestAnimationFrame` and the frame loop.**

**05-uniforms-animation.js**

```
import { initGPU } from "./common.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context, format } = await initGPU(canvas);

const vertices = new Float32Array([
  -0.5, -0.5,  1, 0, 0,
   0.5, -0.5,  0, 1, 0,
   0.5,  0.5,  0, 0, 1,
  -0.5,  0.5,  1, 1, 0,
]);
const indices = new Uint16Array([0, 1, 2, 0, 2, 3]);

const vertexBuffer = device.createBuffer({
  size: vertices.byteLength,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(vertexBuffer, 0, vertices);
const indexBuffer = device.createBuffer({
  size: indices.byteLength,
  usage: GPUBufferUsage.INDEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(indexBuffer, 0, indices);

// Uniform buffer: { time: f32, aspect: f32 }  (8 bytes; we allocate 16)
const uniformData = new Float32Array(4);
const uniformBuffer = device.createBuffer({
  label: "uniforms",
  size: uniformData.byteLength,
  usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
});

const shader = device.createShaderModule({
  code: /* wgsl */ `
    struct Uniforms {
      time: f32,
      aspect: f32,
    };
    @group(0) @binding(0) var<uniform> u: Uniforms;

    struct VSOut {
      @builtin(position) pos: vec4f,
      @location(0) color: vec3f,
    };
```

```
    @vertex
    fn vs(@location(0) position: vec2f,
          @location(1) color: vec3f) -> VSOut {
      let c = cos(u.time);
      let s = sin(u.time);
      let rotated = vec2f(position.x * c - position.y * s,
                          position.x * s + position.y * c);
      var output: VSOut;
      // divide x by aspect so the square stays square on a wide canvas
      output.pos = vec4f(rotated.x / u.aspect, rotated.y, 0.0, 1.0);
      output.color = color;
      return output;
    }

    @fragment
    fn fs(input: VSOut) -> @location(0) vec4f {
      let pulse = 0.75 + 0.25 * sin(u.time * 3.0);
      return vec4f(input.color * pulse, 1.0);
    }
  `,
});

const pipeline = device.createRenderPipeline({
  layout: "auto",
  vertex: {
    module: shader, entryPoint: "vs",
    buffers: [{
      arrayStride: 20,
      attributes: [
        { shaderLocation: 0, offset: 0, format: "float32x2" },
        { shaderLocation: 1, offset: 8, format: "float32x3" },
      ],
    }],
  },
  fragment: { module: shader, entryPoint: "fs", targets: [{ format }] },
});

// The bind group connects our actual buffer to @group(0) @binding(0)
const bindGroup = device.createBindGroup({
  layout: pipeline.getBindGroupLayout(0),
  entries: [{ binding: 0, resource: { buffer: uniformBuffer } }],
});
```

```
function frame(nowMs) {
  uniformData[0] = nowMs / 1000;                  // time in seconds
  uniformData[1] = canvas.width / canvas.height;  // aspect ratio
  device.queue.writeBuffer(uniformBuffer, 0, uniformData);

  const encoder = device.createCommandEncoder();
  const pass = encoder.beginRenderPass({
    colorAttachments: [{
      view: context.getCurrentTexture().createView(),  // fresh view EVERY frame
      clearValue: { r: 0.08, g: 0.08, b: 0.12, a: 1 },
      loadOp: "clear", storeOp: "store",
    }],
  });
  pass.setPipeline(pipeline);
  pass.setBindGroup(0, bindGroup);
  pass.setVertexBuffer(0, vertexBuffer);
  pass.setIndexBuffer(indexBuffer, "uint16");
  pass.drawIndexed(6);
  pass.end();
  device.queue.submit([encoder.finish()]);

  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

## What to notice

- The WGSL `Uniforms` struct has two `f32`: `time` and `aspect`. The JS `Float32Array` mirrors it.

- `@group(0) @binding(0) var<uniform> u` is connected to the real buffer through a bind group whose layout comes from `pipeline.getBindGroupLayout(0)`.
- Each frame: write new values with `writeBuffer`, record, submit. The vertex shader rotates the quad using `cos/sin` of `u.time`; the fragment shader pulses the brightness.
- Dividing x by the aspect ratio prevents the square from stretching on a 4:3 canvas.
- Expected result: a colorful square spinning and gently pulsing.

# 16. Example 06: Textures and samplers

**Goal: display image data. Introduces textures, `writeTexture`, samplers and UV coordinates.**

**06-textures.js**

```
import { initGPU } from "./common.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context, format } = await initGPU(canvas);

// ---- 1. Make an 8x8 checkerboard on the CPU (RGBA, 1 byte per channel)
const size = 8;
const pixels = new Uint8Array(size * size * 4);
for (let y = 0; y < size; y++) {
  for (let x = 0; x < size; x++) {
    const i = (y * size + x) * 4;
    const on = (x + y) % 2 === 0;
    pixels[i + 0] = on ? 240 : 30;
    pixels[i + 1] = on ? 200 : 60;
    pixels[i + 2] = on ? 40 : 160;
    pixels[i + 3] = 255;
  }
}

// ---- 2. Create the GPU texture and upload the pixels
const texture = device.createTexture({
  label: "checkerboard",
  size: [size, size],
  format: "rgba8unorm",
  usage: GPUTextureUsage.TEXTURE_BINDING | GPUTextureUsage.COPY_DST,
});
device.queue.writeTexture(
  { texture },
  pixels,
  { bytesPerRow: size * 4 },
  [size, size]
);

// ---- 3. A sampler says HOW to read the texture
const sampler = device.createSampler({
  magFilter: "nearest",       // crisp pixels when magnified
  minFilter: "nearest",
  addressModeU: "repeat",     // uv > 1 wraps around (tiling)
  addressModeV: "repeat",
});

// x, y, u, v   (uv goes 0..4, so the texture repeats 4 times)
const vertices = new Float32Array([
  -0.8, -0.8,  0, 4,
```

```
   0.8, -0.8,  4, 4,
   0.8,  0.8,  4, 0,
  -0.8,  0.8,  0, 0,
]);
const indices = new Uint16Array([0, 1, 2, 0, 2, 3]);
const vertexBuffer = device.createBuffer({
  size: vertices.byteLength,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(vertexBuffer, 0, vertices);
const indexBuffer = device.createBuffer({
  size: indices.byteLength,
  usage: GPUBufferUsage.INDEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(indexBuffer, 0, indices);

const shader = device.createShaderModule({
  code: /* wgsl */ `
    struct VSOut {
      @builtin(position) pos: vec4f,
      @location(0) uv: vec2f,
    };

    @vertex
    fn vs(@location(0) position: vec2f,
          @location(1) uv: vec2f) -> VSOut {
      var output: VSOut;
      output.pos = vec4f(position, 0.0, 1.0);
      output.uv = uv;
      return output;
    }

    @group(0) @binding(0) var samp: sampler;
    @group(0) @binding(1) var tex: texture_2d<f32>;

    @fragment
    fn fs(input: VSOut) -> @location(0) vec4f {
      return textureSample(tex, samp, input.uv);
    }
  `,
});

const pipeline = device.createRenderPipeline({
  layout: "auto",
```

```
  vertex: {
    module: shader, entryPoint: "vs",
    buffers: [{
      arrayStride: 16,
      attributes: [
        { shaderLocation: 0, offset: 0, format: "float32x2" },
        { shaderLocation: 1, offset: 8, format: "float32x2" },
      ],
    }],
  },
  fragment: { module: shader, entryPoint: "fs", targets: [{ format }] },
});

const bindGroup = device.createBindGroup({
  layout: pipeline.getBindGroupLayout(0),
  entries: [
    { binding: 0, resource: sampler },
    { binding: 1, resource: texture.createView() },
  ],
});

const encoder = device.createCommandEncoder();
const pass = encoder.beginRenderPass({
  colorAttachments: [{
    view: context.getCurrentTexture().createView(),
    clearValue: { r: 0.05, g: 0.05, b: 0.08, a: 1 },
    loadOp: "clear", storeOp: "store",
  }],
});
pass.setPipeline(pipeline);
pass.setBindGroup(0, bindGroup);
pass.setVertexBuffer(0, vertexBuffer);
pass.setIndexBuffer(indexBuffer, "uint16");
pass.drawIndexed(6);
pass.end();
device.queue.submit([encoder.finish()]);
```

## What to notice

- We generate an 8x8 checkerboard in JavaScript, so no image file is needed.
- The texture needs `TEXTURE_BINDING` (shader can sample it) and `COPY_DST` (we upload into it).
- UV coordinates 0..4 make the texture repeat four times thanks to `addressMode: "repeat"`; `nearest` filtering keeps pixels sharp.
- In WGSL the texture and sampler are separate bindings; `textureSample(tex, samp, uv)` combines them. Bind group entries: sampler at binding 0, texture *view* at binding 1.
- Expected result: a tiled yellow/blue checker pattern.

# 17. Example 07: A rotating 3D cube with depth testing

**Goal: real 3D. Introduces 3D vertices, matrices, the MVP uniform, culling and the depth buffer.**

**math.js**

```
// math.js - tiny column-major 4x4 matrix helpers (matches WGSL mat4x4f)
export const mat4 = {
  identity() {
    return new Float32Array([1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1]);
  },
  // out = a * b  (apply b first, then a)
  multiply(a, b) {
    const out = new Float32Array(16);
    for (let col = 0; col < 4; col++) {
      for (let row = 0; row < 4; row++) {
        let sum = 0;
        for (let k = 0; k < 4; k++) sum += a[k * 4 + row] * b[col * 4 + k];
        out[col * 4 + row] = sum;
      }
    }
    return out;
  },
  // Perspective for WebGPU clip space: depth range is 0..1 (not -1..1 like OpenGL)
  perspective(fovY, aspect, near, far) {
    const f = 1 / Math.tan(fovY / 2);
    const m = new Float32Array(16);
    m[0] = f / aspect;
    m[5] = f;
    m[10] = far / (near - far);
    m[11] = -1;
    m[14] = (near * far) / (near - far);
    return m;
  },
  translation(x, y, z) {
    const m = mat4.identity();
    m[12] = x; m[13] = y; m[14] = z;
    return m;
  },
  rotationX(a) {
    const c = Math.cos(a), s = Math.sin(a);
    return new Float32Array([1,0,0,0, 0,c,s,0, 0,-s,c,0, 0,0,0,1]);
  },
  rotationY(a) {
    const c = Math.cos(a), s = Math.sin(a);
    return new Float32Array([c,0,-s,0, 0,1,0,0, s,0,c,0, 0,0,0,1]);
  },
};
```

**07-cube-3d.js**

```
import { initGPU } from "./common.js";
import { mat4 } from "./math.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context, format } = await initGPU(canvas);

// 8 corners: x, y, z, r, g, b
const vertices = new Float32Array([
  -1, -1,  1,  0.2, 0.2, 1.0,   // 0
   1, -1,  1,  1.0, 0.2, 0.2,   // 1
   1,  1,  1,  0.2, 1.0, 0.2,   // 2
  -1,  1,  1,  1.0, 1.0, 0.2,   // 3
  -1, -1, -1,  1.0, 0.2, 1.0,   // 4
   1, -1, -1,  0.2, 1.0, 1.0,   // 5
   1,  1, -1,  1.0, 1.0, 1.0,   // 6
  -1,  1, -1,  0.5, 0.5, 0.5,   // 7
]);
// 12 triangles, every one counter-clockwise when seen from OUTSIDE the cube
const indices = new Uint16Array([
  0, 1, 2,  0, 2, 3,    // front  (+z)
  5, 4, 7,  5, 7, 6,    // back   (-z)
  1, 5, 6,  1, 6, 2,    // right  (+x)
  4, 0, 3,  4, 3, 7,    // left   (-x)
  3, 2, 6,  3, 6, 7,    // top    (+y)
  4, 5, 1,  4, 1, 0,    // bottom (-y)
]);

const vertexBuffer = device.createBuffer({
  size: vertices.byteLength,
  usage: GPUBufferUsage.VERTEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(vertexBuffer, 0, vertices);
const indexBuffer = device.createBuffer({
  size: indices.byteLength,
  usage: GPUBufferUsage.INDEX | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(indexBuffer, 0, indices);

// One mat4x4f = 16 floats = 64 bytes
const uniformBuffer = device.createBuffer({
  size: 64,
  usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
});
```

```
// The depth buffer decides which surface is in front at each pixel
const depthTexture = device.createTexture({
  size: [canvas.width, canvas.height],
  format: "depth24plus",
  usage: GPUTextureUsage.RENDER_ATTACHMENT,
});

const shader = device.createShaderModule({
  code: /* wgsl */ `
    struct Uniforms { mvp: mat4x4f };
    @group(0) @binding(0) var<uniform> u: Uniforms;

    struct VSOut {
      @builtin(position) pos: vec4f,
      @location(0) color: vec3f,
    };

    @vertex
    fn vs(@location(0) position: vec3f,
          @location(1) color: vec3f) -> VSOut {
      var output: VSOut;
      output.pos = u.mvp * vec4f(position, 1.0);
      output.color = color;
      return output;
    }

    @fragment
    fn fs(input: VSOut) -> @location(0) vec4f {
      return vec4f(input.color, 1.0);
    }
  `,
});

const pipeline = device.createRenderPipeline({
  layout: "auto",
  vertex: {
    module: shader, entryPoint: "vs",
    buffers: [{
      arrayStride: 6 * 4,
      attributes: [
        { shaderLocation: 0, offset: 0,      format: "float32x3" },
        { shaderLocation: 1, offset: 3 * 4,  format: "float32x3" },
      ],
    }],
```

```
  },
  fragment: { module: shader, entryPoint: "fs", targets: [{ format }] },
  primitive: { topology: "triangle-list", cullMode: "back" }, // skip back faces
  depthStencil: {
    format: "depth24plus",
    depthWriteEnabled: true,
    depthCompare: "less",          // nearer fragments win
  },
});

const bindGroup = device.createBindGroup({
  layout: pipeline.getBindGroupLayout(0),
  entries: [{ binding: 0, resource: { buffer: uniformBuffer } }],
});

const projection = mat4.perspective(
  Math.PI / 4, canvas.width / canvas.height, 0.1, 100
);
const view = mat4.translation(0, 0, -5);       // move the world away from camera

function frame(nowMs) {
  const t = nowMs / 1000;
  const model = mat4.multiply(mat4.rotationX(t * 0.7), mat4.rotationY(t));
  // MVP = Projection * View * Model   (rightmost is applied first)
  const mvp = mat4.multiply(projection, mat4.multiply(view, model));
  device.queue.writeBuffer(uniformBuffer, 0, mvp);

  const encoder = device.createCommandEncoder();
  const pass = encoder.beginRenderPass({
    colorAttachments: [{
      view: context.getCurrentTexture().createView(),
      clearValue: { r: 0.06, g: 0.06, b: 0.1, a: 1 },
      loadOp: "clear", storeOp: "store",
    }],
    depthStencilAttachment: {
      view: depthTexture.createView(),
      depthClearValue: 1.0,
      depthLoadOp: "clear",
      depthStoreOp: "store",
    },
  });
  pass.setPipeline(pipeline);
  pass.setBindGroup(0, bindGroup);
  pass.setVertexBuffer(0, vertexBuffer);
```

```
  pass.setIndexBuffer(indexBuffer, "uint16");
  pass.drawIndexed(indices.length);
  pass.end();
  device.queue.submit([encoder.finish()]);
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

## What to notice

- `math.js` (below) provides column-major matrix helpers. `mat4.perspective` produces WebGPU-style depth 0..1.
- The cube has 8 shared corners and 36 indices. Winding is counter-clockwise as seen from outside, so `cullMode: "back"` removes hidden faces.
- The uniform buffer holds one `mat4x4f` (64 bytes). Each frame we compute `projection * view * model` and upload it.
- The depth texture must be as large as the canvas and used as `depthStencilAttachment`; the pipeline declares the same format in `depthStencil`.
- Expected result: a multicolor cube rotating around two axes with correct occlusion.

# 18. Example 08: Compute shader: process a million numbers on the GPU

**Goal: your first compute shader. Introduces storage buffers, compute pipelines, dispatch, copy to a staging buffer and `mapAsync` readback.**

**08-compute-double.js**

```
import { initGPU, log } from "./common.js";

const { device } = await initGPU();

const N = 1_000_000;
const input = new Float32Array(N);
for (let i = 0; i < N; i++) input[i] = i;
const byteSize = input.byteLength;

// Buffer the shader reads AND writes
const storageBuffer = device.createBuffer({
  label: "data",
  size: byteSize,
  usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_SRC | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(storageBuffer, 0, input);

// Storage buffers cannot be mapped to the CPU, so we copy into a
// "staging" buffer that CAN be mapped for reading.
const readBuffer = device.createBuffer({
  label: "readback",
  size: byteSize,
  usage: GPUBufferUsage.MAP_READ | GPUBufferUsage.COPY_DST,
});

const shader = device.createShaderModule({
  code: /* wgsl */ `
    @group(0) @binding(0) var<storage, read_write> data: array<f32>;

    @compute @workgroup_size(64)
    fn main(@builtin(global_invocation_id) id: vec3u) {
      let i = id.x;
      if (i >= arrayLength(&data)) { return; }   // guard the last workgroup
      data[i] = data[i] * 2.0;
    }
  `,
});

const pipeline = device.createComputePipeline({
  layout: "auto",
  compute: { module: shader, entryPoint: "main" },
});
const bindGroup = device.createBindGroup({
  layout: pipeline.getBindGroupLayout(0),
```

```
  entries: [{ binding: 0, resource: { buffer: storageBuffer } }],
});

const encoder = device.createCommandEncoder();
const pass = encoder.beginComputePass();
pass.setPipeline(pipeline);
pass.setBindGroup(0, bindGroup);
pass.dispatchWorkgroups(Math.ceil(N / 64));      // 15,625 workgroups x 64 threads
pass.end();
encoder.copyBufferToBuffer(storageBuffer, 0, readBuffer, 0, byteSize);
device.queue.submit([encoder.finish()]);

// Wait for the GPU, then read the result
await readBuffer.mapAsync(GPUMapMode.READ);
const result = new Float32Array(readBuffer.getMappedRange().slice(0));
readBuffer.unmap();

log("first 5 :", Array.from(result.slice(0, 5)).join(", "));   // 0, 2, 4, 6, 8
log("last 3  :", Array.from(result.slice(-3)).join(", "));
log("expected:", (N - 1) * 2, "for the last element");
```

## What to notice

- One million floats (4 MB) are uploaded, doubled on the GPU by 15,625 workgroups of 64 invocations each, then read back.
- `var<storage, read_write>` plus `arrayLength(&data)` gives a runtime-sized array view of the whole buffer.
- The guard `if (i >= arrayLength(&data)) return;` handles the extra invocations in the last workgroup.
- We cannot map a `STORAGE` buffer directly, so we copy it into a `MAP_READ | COPY_DST` buffer. `getMappedRange().slice(0)` makes a copy that stays valid after `unmap()`.
- Expected result: `0, 2, 4, 6, 8` for the first five and `1999998` for the last element.

# 19. Example 09: GPU particle system: compute + instanced rendering

**Goal: bring everything together. Twenty thousand particles are simulated by a compute shader and drawn by a render pipeline, all on the GPU.**

**09-gpu-particles.js**

```
import { initGPU } from "./common.js";

const canvas = document.getElementById("gpu-canvas");
const { device, context, format } = await initGPU(canvas);

const N = 20000;                       // number of particles

// Each particle: pos (vec2f) + vel (vec2f) = 4 floats = 16 bytes
const init = new Float32Array(N * 4);
for (let i = 0; i < N; i++) {
  init[i * 4 + 0] = Math.random() * 2 - 1;        // pos.x
  init[i * 4 + 1] = Math.random() * 2 - 1;        // pos.y
  init[i * 4 + 2] = (Math.random() - 0.5) * 0.6;  // vel.x
  init[i * 4 + 3] = (Math.random() - 0.5) * 0.6;  // vel.y
}
const particleBuffer = device.createBuffer({
  label: "particles",
  size: init.byteLength,
  usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_DST,
});
device.queue.writeBuffer(particleBuffer, 0, init);

const paramsBuffer = device.createBuffer({
  label: "params",
  size: 16,
  usage: GPUBufferUsage.UNIFORM | GPUBufferUsage.COPY_DST,
});

// ---------- compute shader: move particles, bounce off the walls
const computeModule = device.createShaderModule({
  code: /* wgsl */ `
    struct Particle { pos: vec2f, vel: vec2f };
    struct Params { dt: f32 };

    @group(0) @binding(0) var<storage, read_write> particles: array<Particle>;
    @group(0) @binding(1) var<uniform> params: Params;

    @compute @workgroup_size(64)
    fn main(@builtin(global_invocation_id) gid: vec3u) {
      let i = gid.x;
      if (i >= arrayLength(&particles)) { return; }

      var p = particles[i];
      p.pos = p.pos + p.vel * params.dt;
```

```
      if (abs(p.pos.x) > 1.0) {
        p.vel.x = -p.vel.x;
        p.pos.x = clamp(p.pos.x, -1.0, 1.0);
      }
      if (abs(p.pos.y) > 1.0) {
        p.vel.y = -p.vel.y;
        p.pos.y = clamp(p.pos.y, -1.0, 1.0);
      }
      particles[i] = p;
    }
  `,
});

// ---------- render shader: draw each particle as a tiny instanced quad
const renderModule = device.createShaderModule({
  code: /* wgsl */ `
    struct Particle { pos: vec2f, vel: vec2f };
    @group(0) @binding(0) var<storage, read> particles: array<Particle>;

    struct VSOut {
      @builtin(position) pos: vec4f,
      @location(0) color: vec3f,
    };

    @vertex
    fn vs(@builtin(vertex_index) vi: u32,
          @builtin(instance_index) ii: u32) -> VSOut {
      var corners = array<vec2f, 6>(
        vec2f(-1, -1), vec2f(1, -1), vec2f(1, 1),
        vec2f(-1, -1), vec2f(1, 1),  vec2f(-1, 1)
      );
      let p = particles[ii];
      var output: VSOut;
      output.pos = vec4f(p.pos + corners[vi] * 0.006, 0.0, 1.0);
      output.color = vec3f(0.5 + 0.5 * p.pos.x, 0.5 + 0.5 * p.pos.y, 1.0);
      return output;
    }

    @fragment
    fn fs(input: VSOut) -> @location(0) vec4f {
      return vec4f(input.color, 1.0);
    }
  `,
```

```
});

const computePipeline = device.createComputePipeline({
  layout: "auto",
  compute: { module: computeModule, entryPoint: "main" },
});
const renderPipeline = device.createRenderPipeline({
  layout: "auto",
  vertex:   { module: renderModule, entryPoint: "vs" },
  fragment: { module: renderModule, entryPoint: "fs", targets: [{ format }] },
});

// Same particle buffer, bound into two different pipelines
const computeBG = device.createBindGroup({
  layout: computePipeline.getBindGroupLayout(0),
  entries: [
    { binding: 0, resource: { buffer: particleBuffer } },
    { binding: 1, resource: { buffer: paramsBuffer } },
  ],
});
const renderBG = device.createBindGroup({
  layout: renderPipeline.getBindGroupLayout(0),
  entries: [{ binding: 0, resource: { buffer: particleBuffer } }],
});

let last = performance.now();
function frame(now) {
  const dt = Math.min((now - last) / 1000, 0.05);   // clamp huge gaps
  last = now;
  device.queue.writeBuffer(paramsBuffer, 0, new Float32Array([dt]));

  const encoder = device.createCommandEncoder();

  const cpass = encoder.beginComputePass();          // 1) simulate
  cpass.setPipeline(computePipeline);
  cpass.setBindGroup(0, computeBG);
  cpass.dispatchWorkgroups(Math.ceil(N / 64));
  cpass.end();

  const rpass = encoder.beginRenderPass({            // 2) draw
    colorAttachments: [{
      view: context.getCurrentTexture().createView(),
      clearValue: { r: 0, g: 0, b: 0.03, a: 1 },
      loadOp: "clear", storeOp: "store",
    }],
  });
  rpass.setPipeline(renderPipeline);
  rpass.setBindGroup(0, renderBG);
  rpass.draw(6, N);                                  // 6 vertices x N instances
  rpass.end();

  device.queue.submit([encoder.finish()]);           // ONE submit per frame
  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
```

## What to notice

- The particle data never returns to the CPU: one storage buffer is written by the compute pipeline and read by the render pipeline's vertex shader.
- Two bind groups share the same buffer (different pipelines have different layouts, so each gets its own bind group).
- Rendering uses **instancing**: `draw(6, N)` draws a 6-vertex quad N times; `instance_index` selects which particle to read.
- `dt` (time since last frame) is passed in a small uniform buffer so the movement is frame-rate independent.

- One command encoder holds a compute pass followed by a render pass, then **one** `submit`. Order inside the encoder guarantees the render pass sees the updated particles.
- Expected result: a field of colored dots drifting and bouncing. Try raising N to 200000.

# 20. Debugging, pitfalls and best practices

## 20.1 When nothing shows up

| Symptom | Likely cause and fix |
|---|---|
| Black / empty canvas, no error | Forgot `pass.end()`, `queue.submit()`, or `draw()`. Check that the vertex positions are inside -1..1. Try `cullMode: "none"`. |
| Nothing drawn after resize | Canvas has size 0, or depth texture size no longer matches the canvas. Recreate size-dependent textures on resize. |
| `navigator.gpu` is undefined | Browser too old or page not a secure context (use `localhost` or HTTPS). |
| `requestAdapter()` returns null | GPU blocklisted, hardware acceleration disabled, or remote session. Check the browser's GPU status page. |
| Validation error about usage | Buffer/texture missing a usage flag (e.g. `COPY_DST` for `writeBuffer`, `VERTEX` to bind as vertex buffer). |
| Validation error about size | `writeBuffer` size/offset not a multiple of 4, or the buffer is smaller than the struct in the shader. |
| Pipeline target mismatch | Pipeline `targets[0].format` differs from the canvas format. Use `getPreferredCanvasFormat()` for both. |
| Garbage values in uniforms | Struct alignment (vec3!). Compare the JS offsets against the WGSL layout table in chapter 5.6. |
| Texture / current frame error | Cached `getCurrentTexture()` from a previous frame. Call it again every frame. |
| Readback returns zeros | Forgot `COPY_SRC` on the source, forgot the copy command, or read before `await mapAsync`. |
| Everything inside-out in 3D | Triangle winding reversed relative to `frontFace`, or matrix multiplication order wrong (it is P * V * M). |

## 20.2 Debugging tools

- Give every object a `label`: error messages then name the culprit.
- Wrap risky code in `pushErrorScope` / `popErrorScope`.
- Call `shader.getCompilationInfo()` to print WGSL errors with line and column.
- Use the browser devtools console; Chrome also has a WebGPU inspector extension and the Spector-style tools from the community.
- Isolate: replace the fragment output with a constant color, then with interpolated values, to find which stage is wrong.

**error handling**

```
device.pushErrorScope("validation");
const pipeline = device.createRenderPipeline({ /* ... */ });
const err = await device.popErrorScope();
if (err) console.error("Pipeline problem:", err.message);

const info = await shader.getCompilationInfo();
for (const m of info.messages) {
  console.warn(`${m.type} line ${m.lineNum}:${m.linePos} ${m.message}`);
}
```

## 20.3 Handling window resize and high-DPI screens

**resize pattern**

```
function resize() {
  const dpr = window.devicePixelRatio || 1;
  const w = Math.max(1, Math.floor(canvas.clientWidth * dpr));
  const h = Math.max(1, Math.floor(canvas.clientHeight * dpr));
  if (canvas.width !== w || canvas.height !== h) {
    canvas.width = w;
    canvas.height = h;
    depthTexture?.destroy();
    depthTexture = device.createTexture({
      size: [w, h], format: "depth24plus",
      usage: GPUTextureUsage.RENDER_ATTACHMENT,
    });
    // also update projection matrix with the new aspect ratio
  }
}
new ResizeObserver(resize).observe(canvas);
```

## 20.4 Loading a real image as a texture

**image-texture.js**

```
const response = await fetch("photo.png");
const bitmap = await createImageBitmap(await response.blob());

const texture = device.createTexture({
  size: [bitmap.width, bitmap.height],
  format: "rgba8unorm",
  usage: GPUTextureUsage.TEXTURE_BINDING |
         GPUTextureUsage.COPY_DST |
         GPUTextureUsage.RENDER_ATTACHMENT,   // required by copyExternalImage...
});
device.queue.copyExternalImageToTexture(
  { source: bitmap },
  { texture },
  [bitmap.width, bitmap.height]
);
```

## 20.5 Performance habits

- **Create once, reuse:** pipelines, bind groups, buffers and textures at load time; per frame only write data and record commands.
- **One submit per frame**, with several passes in one encoder.
- **Batch small uniforms** into one buffer, and use dynamic offsets or storage buffers for per-object data instead of hundreds of tiny buffers.
- **Avoid per-frame readback.** If you need results on the CPU, read them a frame or two late.
- **Use async pipeline creation** for big shaders and warm them up during loading.
- **Use render bundles** (`createRenderBundleEncoder`) for static scenes that you draw the same way every frame.
- **Pick sensible workgroup sizes** (64 or 256) and keep memory access patterns coalesced: neighbouring invocations should touch neighbouring addresses.
- **Use** `destroy()` on large textures and buffers when you are done to free GPU memory promptly.
- Profile on real devices: integrated GPUs and phones behave very differently from desktop cards. The optional `timestamp-query` feature measures pass durations on the GPU.

## 20.6 Cheat sheet

| Task | Code |
|---|---|
| Get device | `const adapter = await navigator.gpu.requestAdapter(); const device = await adapter.requestDevice();` |
| Configure canvas | `context.configure({ device, format: navigator.gpu.getPreferredCanvasFormat() })` |
| Create buffer | `device.createBuffer({ size, usage: GPUBufferUsage.VERTEX \| GPUBufferUsage.COPY_DST })` |
| Upload | `device.queue.writeBuffer(buffer, 0, typedArray)` |
| Shader | `device.createShaderModule({ code })` |
| Render pipeline | `device.createRenderPipeline({ layout: "auto", vertex, fragment, primitive, depthStencil })` |
| Compute pipeline | `device.createComputePipeline({ layout: "auto", compute: { module, entryPoint } })` |
| Bind group | `device.createBindGroup({ layout: pipeline.getBindGroupLayout(0), entries: [...] })` |
| Record | `encoder = device.createCommandEncoder(); pass = encoder.beginRenderPass({...}); ... pass.end()` |
| Submit | `device.queue.submit([encoder.finish()])` |
| Draw | `pass.setPipeline(p); pass.setBindGroup(0, bg); pass.setVertexBuffer(0, vb); pass.draw(n)` |
| Dispatch | `pass.dispatchWorkgroups(Math.ceil(N / 64))` |
| Readback | `copyBufferToBuffer` -> `await buf.mapAsync(GPUMapMode.READ)` -> `getMappedRange()` -> `unmap()` |

# 21. Where to go next

You now know every core concept. The following topics build directly on what you learned, in a sensible order:

- **Lighting:** pass normals and a light direction; compute diffuse and specular terms in the fragment shader (Lambert, Blinn-Phong), then move to PBR.
- **Multiple objects:** store per-object transforms in a storage buffer and draw with instancing; add a camera controller (orbit controls).
- **Loading models:** parse glTF files and upload their vertex and index buffers.
- **Multisampling (MSAA):** create a multisampled color texture, set `multisample: { count: 4 }` in the pipeline and resolve to the canvas texture.
- **Mipmaps and filtering:** generate mip levels for smooth minification, use `linear` filtering and anisotropy.
- **Post-processing:** render the scene to a texture, then a second pass reads that texture to apply blur, bloom, color grading.
- **Shadow mapping:** render depth from the light's point of view, then compare depths when shading.
- **Advanced compute:** parallel reductions, prefix sums, N-body simulation, fluid simulation, image convolution, and machine-learning inference kernels.
- **Render bundles, indirect drawing and GPU-driven rendering** for very large scenes.

## 21.1 Good references

- **WebGPU Fundamentals** (webgpufundamentals.org): step-by-step lessons covering everything here and much more.
- **MDN Web Docs, WebGPU API section:** the reference for every object and method.
- **The W3C WebGPU specification** (gpuweb.github.io/gpuweb) and **the WGSL specification** (w3.org/TR/WGSL): exact rules on layouts, limits and semantics.
- **Google's "Your first WebGPU app" codelab** and **Toji's WebGPU best-practices** articles for performance guidance.
- **Three.js (WebGPURenderer) and Babylon.js:** read how production engines structure pipelines and bind groups.

## 21.2 Glossary

| Term | Meaning |
|---|---|
| Adapter | Object describing a physical GPU (or software renderer) |
| Device | Your logical connection to the GPU; creates all other objects |
| Queue | Where command buffers and uploads are submitted |
| Command encoder / buffer | Recorder of GPU commands / the finished, submittable recording |
| Pipeline | Immutable bundle of shaders plus fixed-function state |
| Bind group (layout) | Set of resources bound together (and the description of its shape) |
| WGSL | WebGPU Shading Language |
| Vertex / fragment shader | Program run per vertex / per covered pixel |
| Compute shader | General-purpose GPU program over a grid of invocations |
| Workgroup | Group of invocations that can share memory and synchronize |
| Uniform / storage buffer | Small read-only / large read-write buffer visible to shaders |
| Attachment | Texture that a render pass draws into (color or depth) |
| NDC / clip space | Coordinate spaces the vertex shader outputs into (x,y in -1..1; z in 0..1) |
| Instancing | Drawing the same mesh many times in one call |
| Mapping | Giving the CPU access to a buffer's bytes (`mapAsync`, `getMappedRange`, `unmap`) |

**Final advice**

Re-type examples 03, 05 and 08 from memory, then change them: add a second triangle, animate the color with a uniform, make the compute shader square the numbers instead of doubling them. Small experiments are how this knowledge sticks.